

CODENTIC AI · EMERGING WORLD MODELS

A memory side-car beside the model you run

World models are one of the most active directions in AI research. EWM is the part we build: the memory — structural, content-addressed, verifiable, and portable across models.

NO RETRAINING · NO FINE-TUNING · NO NEW WEIGHTS · ON-PREMISE

REFERENCE IMPLEMENTATION: EWM-STATE-MACHINE — RUST WORKSPACE, 15 CRATES, 12 NOTEBOOKS

codenticai.com · github.com/alexmy21/ewm-state-machine



A side-car, in plain words

Start with the motorbike

A side-car is the extra seat bolted to the side of a motorbike. The bike is not taken apart to fit it — the engine, the wheels and the handlebars stay exactly as they were. The side-car travels alongside and adds something the bike never had.

Same idea: a battery phone case, a bike rack on a car, an external drive.

Why people build this way

- Nothing that already works gets opened up, so there is no risk of breaking it.
- The add-on can be fitted, updated or removed on its own; the main thing never changes.
- Each part can be looked after and repaired separately.
- Far quicker and cheaper than rebuilding the main thing.

Software already uses this pattern everywhere — small helpers attached to a large program for logging, security checks or monitoring, so the large program never has to be modified.

Now put your AI model where the motorbike is

Our side-car is bolted on beside the model you already use, and its single job is to **remember**.

It listens to the signals the model produces while it works — its internal encodings — keeps a record of them **outside the model**, and can hand similar signals back when the model needs them.

The model is not opened up, retrained or replaced — it keeps its weights, tokenizer and usual behaviour.

The honest limit: we can only work with the signals the model makes available to us.

Context and memory are two different jobs

Context — the working set

Everything the model can see at the moment it answers: instructions, the conversation so far, retrieved passages, tool output. Re-sent every turn, bounded by the window, gone once the call returns.

Memory — the durable state

What a system keeps after the call is over and can find again on purpose: decisions, results, history, structure. It lives outside the model — in a store, an index, a database — and retrieval is the pipe into context.

	CONTEXT	MEMORY
What it is	The working set visible in a single call	State that survives between calls, addressable later
Lifetime	The length of the call	Until something explicitly removes it
What limits it	The context window	Whatever your storage can hold
Cost behaviour	Re-sent, and re-priced, on every turn	Read when it is needed
How you check it	You can inspect what you sent	Only if the store itself is checkable

The pipeline is **memory** → **retrieval** → **context**. A system can have a very large window and still have no memory — and adding retrieval does not by itself produce a memory you can audit.

The missing piece is memory, not intelligence

1 Bounded working context

Reasoning quality is capped by what fits in one window. Everything relevant must be re-sent every turn, so cost and latency scale with it. When the window fills, something is dropped — heuristically, not structurally — and the model cannot tell you what it lost.

2 Lossy, unstructured history

Long-term memory arrives as summaries, notes or retrieved chunks. Summarization discards structure — counts, overlaps and novelty vanish. Retrieval is similarity, not position in a lattice, so there is no guarantee of coverage.

3 No structural view at all

You cannot ask a model's own output what is new, what is retained and what departed. There is no algebra over model state — only text diffs and embeddings. Nothing is content-addressed, so nothing is independently verifiable.

How the side-car relates to all three: it does not remove the window, and it does not replace retrieval or summaries. It adds a structural state beside them — the model keeps reasoning exactly as it does today, and gains a record of what it retained, what departed and what is new.

Where EWM sits in the world-model movement

World models are a bet much of the field is making — that a system should carry an internal, predictive model of its world, not only a larger next-token predictor. Latent predictive architectures such as JEPA, learned simulators and generative video models are all part of that programme, advanced by far more groups than ours. **We do not claim the thesis. We contribute one part of it: the memory.**

A shared programme, not a contested one

We treat the wider world-model work as the ground EWM grows in. Our papers, vocabulary and guarantees are meant to sit alongside it — a layer other groups can use, including teams building predictive encoders and latent state models.

Model-agnostic on purpose

Because EWM works at the measurement level, one lattice spans model families and modalities: a JEPA-style predictive encoder, a vision stack, an OCR model or a driving policy. The memory travels; the model stays where it is.

The side-car sits on the **memory side**, not the context side: a home for durable state outside the model, with a structure that can be checked — what is retained, what departed and what is new.

We add to your model, we do not replace it

The side-car reads the host's own internal encodings, keeps state outside the model and hands encodings back. Weights, tokenizer, prompts and APIs are untouched — adopting the memory never means retiring the model.

What we do claim

That a memory layer beside a model can be **measured, verified and carried across models** — with no change to the model itself. A deliberately narrow claim, and the evidence for it is published in full.

Six properties — each traceable to a run, not asserted



A structure, not a summary

D / R / N per commit — departed, retained, novel — with a conservation balance. A precise answer to "what changed?"



The model is untouched

Zero model changes — no retraining, no fine-tuning, no new weights. The host keeps its weights, tokenizer and APIs.



Auditable by construction

Content-addressed, immutable commits give an append-only record — exactly what an auditor asks for.



Portable across models

Measurement-level and model-agnostic: the same lattice spans hosts and modalities, so memory is not trapped in one checkpoint.



On-premise & air-gapped

Deployed inside your own boundary, including air-gapped environments with zero outbound calls. The state store sits in storage you control.



Proven on real models

Six measured scenarios across a vision-language-action stack, OCR, vision grids and a document platform.

Bring your own model — we bring the memory. Your weights, tokenizer and APIs stay exactly as they are.

Measured on real models, caveats published

Every figure comes from a run in our own repositories — real model encodings, real documents, on our own machines. Where a number has a limit, the limit is printed beside it.

1.0

loop accuracy — hash-level round-trip on a real VLA

100%

word retention — ordered OCR reconstruction

32,768

measurement sites per state, content-addressed

0

model changes — no retraining, no fine-tuning, no new weights

All six measured scenarios

- **Vision-language-action stack** — hash-level round-trip accuracy 1.0 on a real VLA host
- **Document intelligence (DeepSeek-OCR)** — 100% word retention, zero order loss, across a real 128K-vocabulary BPE boundary
- **Vision grids** — 16×16 patch grids through conv(n,2) encoders, frame-by-frame positional restoration
- **World-model memory (JEPA-style)** — content-addressed recall by Bell State Similarity, no GPU, no torch
- **Language models** — multi-LLM side-car loops on real small LLMs · **Document platform** — the semantic compressor

Clinical and diagnostic outputs are research demonstrations for partner evaluation — not a medical device, and no replacement for clinician review.

Under the hood: every state is a set of bits

The memory does not store text or embeddings. Each signal the model emits is lifted — by a fixed, deterministic mapping — into a **large bit set** called an **HLLSet**: a row of positions, each either on or off.

Because every state is a set of bits over the same space, states can be compared with plain set operations — union, intersection, difference — at the bit level. That is what makes the memory **structural**: "what changed?" becomes a set difference, not a guess over text.

The mapping itself is the only contract, called **IICA** — Idempotence, Immutability, Content Addressability: the same input always yields the same bit set, states never change once written, and every state is named by its own content hash. Any mapping that honours IICA is interchangeable.

Tokens are restored only by **materialization** — a lookup that keeps every candidate, probabilistic on the extras, exact on the builders. Ingest lifts tokens into bits; materialize lowers bits back into tokens.

ONE HLLSET — A ROW OF BITS



SET OPERATIONS OVER THE SAME SPACE

$A \cup B$	union — everything either state holds
$A \cap B$	intersection — what both states share
$A \setminus B$	difference — what one state added or lost

EVERY STATE NAMED BY ITS CONTENT

`h:<sha1> → the state itself`

Same content, same name — so any copy of the memory can be checked against any other, bit for bit.

The fundamental equation — what changed, answered exactly

$$H(t) = (S(t), H(t-1), D, R, N)$$

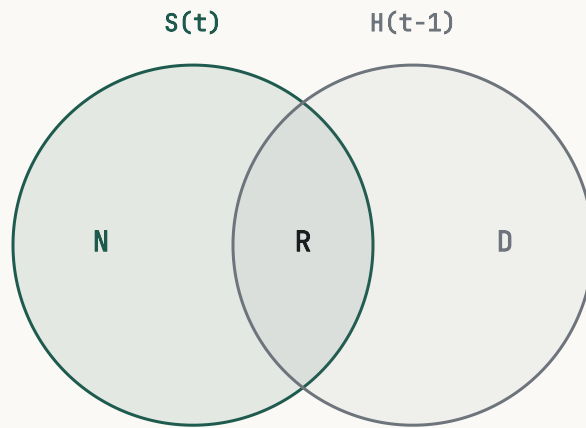
S(t) — the state forming right now; work in progress, the tip of the stack

H(t-1) — the last committed state; the cache, required to commit

N(t) = S(t) \ H(t-1) — what is new

R(t) = S(t) ∩ H(t-1) — what is retained (the R-link)

D(t) = H(t-1) \ S(t) — what departed



D / R / N are derived at every commit — set differences and the intersection over the bit plane

Everything in the equation is an HLLSet, and every operation runs at the bit level. Departed + retained + new, with a conservation balance — a precise answer to "what changed?" on every commit.

Four ideas carry the whole model

1

One universe

Every state is a node of the same Boolean lattice over the bit plane — sets ordered by inclusion, combined by union and intersection. Nothing new is created: a computation names a lattice expression and discovers the node.

2

The morphism is the contract (IICA)

The token $\rightarrow$ bit connection is a chosen mapping μ : token collections $\rightarrow$ bit vectors. Only IICA is required; the bit vector forgets provenance by design, and any IICA algorithm is interchangeable.

3

The fiber is global

The same bit in any HLLSet points to the same lookup-table fiber — the fiber is a property of the bit address, never of the state. Materialization is exact on the builders, probabilistic on the extras.

4

Decompositions are frames (readings)

D/R/N, joined perceptrons, ring basis — each is a named set of dimensions; $\varphi_D(X) = (|X \cap D_i| / |D_i|)$ is the coordinate map, and $\varphi_D(X(t))$ over t is the state trajectory.

ewm-scene makes all four executable: ingest / materialize (2, 3) · noether / subframes (D/R/N frame) · pyramid (joined perceptrons) · sidecar (ring frame) · project (any named frame)

The measurement plate: 4 KB that can reflect 2^{32768} states

Every state the memory can ever hold is a subset of one fixed set of **32,768 measurement sites** — the bit plane. The collection of all such subsets is the lattice: one node per possible state, 2^{32768} nodes in all.

Written out, that is a 9,865-digit number — about 2^{32502} times the number of atoms in the observable universe. No storage can hold the lattice, and no enumeration can list it.

And it never has to. The lattice is **emerging**: it is grown node by node, never built whole. A computation names the nodes it needs — a sketch, a union, an intersection, a residual — and nothing else exists.

The cost of memory is therefore proportional to the **measurements actually taken**, never to the plane they could occupy.

Any model can be brought to the same plate — what differs is where its signals land, not the instrument that holds them.

ONE PLATE, IN NUMBERS

32,768

measurement sites — 1,024 registers × 32 bits, the hllset-contracts default

4 KB

the packed size of any state — after one token or one million

2^{32768}

addressable states on one plate — the 9,865-digit node count

$2^{32502} \times$

larger than the count of atoms in the observable universe

Source: [The EWM Lattice and Its Worlds — companion note](#), Aug 2026 · [Emerging World Models — main paper](#)

Worlds are separated by their LUTs, not by their bit vectors

A **world** is not a copy of the memory — it is a choice of measurement: a hash, a seed and a threshold, $w = (h, s, \tau)$. Change the seed and the same token lands on different sites; both readings still live on the very same plate.

So one lattice carries many worlds at once — several vocabularies, several models, several bootstrap schemes. Each is a configuration, none is a copy.

Worlds may share a state — even the very same stored node — and still read it differently. Each world recovers its tokens through **its own lookup table (LUT)**: $L_w(X) = \cup_{b \in X} L_w[b]$. Overlap on the representation plane is resolved on the **interpretation layer**.

Worlds meet only through a **bridge**: materialize in w_1 , re-ingest in w_2 . The bridge carries encodings, not content.

A new world costs one LUT and a threshold — never another lattice. This virtual isolation is a property of the algebra ("emergence security"), not a wall — and not a substitute for cryptography.

ONE SHARED NODE — TWO READINGS



$h:<sha1>$ — one node, stored once



Same bits, different fibers — a bit address means whatever the world's LUT says it means. A gate is one world's interpretation, restricted: frames do not leak across.

THE HOLOGRAPHIC PROPERTY

One shared representation, a separate interpretation fiber per world — the budget stays 32K bits, shared by every world.

Source: [The EWM Lattice and Its Worlds — companion note](#), Aug 2026 · [Emerging World Models — main paper](#)

Naming 2^{32768} nodes: three hashes where one runs out

Today every state is named by its content hash — a single SHA-1 digest. But SHA-1 is a 160-bit name: it can distinguish at most 2^{160} values, while the plate holds 2^{32768} possible states. No single short hash can name them all — collisions are a mathematical certainty, not a bug.

The fix is an **extendable key**: name each state by three hashes instead of one — $\kappa_3(H) = (\text{SHA1}(H), H_1(H, s_1), H_2(H, s_2))$ — with one lookup table per hash: `sha1-hllsetLUT`, `hash1-hllsetLUT`, `hash2-hllsetLUT`.

Growth is **lazy**: the second table is written only when a SHA-1 collision actually occurs, the third only on a second collision. Almost every lookup still ends at the first table.

The same trick extends to m hashes: full coverage of the plate needs $m = 205$ SHA-1 digests (128 with SHA-256). κ_3 already names 2^{480} states; a mistaken identity is a $\sim 2^{-320}$ event.

THE CAPACITY GAP

2^{160} SHA-1 key space — one digest

2^{32768} nodes of one lattice

2^{480} names under the κ_3 key

LAZY THREE-STAGE RETRIEVAL

1 · `sha1-hllsetLUT` → **singleton: done**

the everyday case — one lookup settles it

2 · + `hash1-hllsetLUT` → **intersect**

only when a real collision occurred

3 · + `hash2-hllsetLUT` → **n of three**

$L_{\text{sha1}} \cap L_{\text{h1}} \cap L_{\text{h2}} = \{H\}$ — exact, no candidates left

Source: [The EWM Lattice and Its Worlds — companion note](#), Aug 2026 · [Emerging World Models — main paper](#)

S(t) is the state in a stateless system

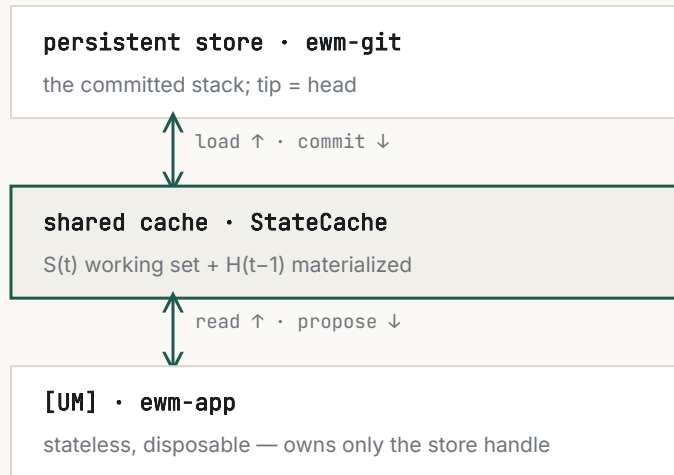
IICA

Idempotence · Immutability · Content Addressability

IICA removes the contradiction: S(t) is an **immutable, content-addressed value**, so it is safe to share.

The **[UM]** — the processing unit that works on S(t) — is **stateless and disposable**: it reads the tip, processes incoming tokens, and proposes the next state.

S(t) and H(t-1) live **outside the [UM]**, in a shareable cache (ewm-app :: StateCache), backed by the persistent store (ewm-git).

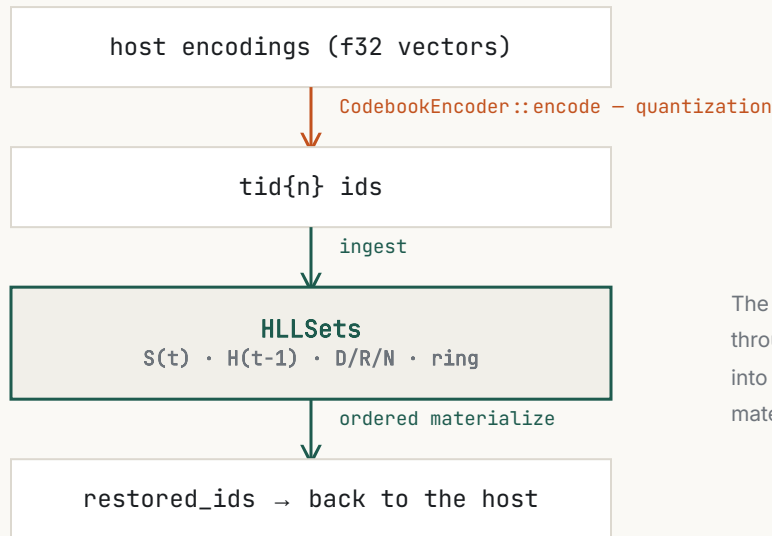


Recovery is a pop, not a rebuild. Drop the broken [UM] and its cache, restore the cache from the tip, and continue with a fresh [UM]. Uncommitted work is reprocessed — idempotent by IICA.

One vocabulary-aware step — everything below it is agnostic

The encoder is the only **vocabulary-aware step** — a shared codebook quantizer (ewm-
app :: CodebookEncoder).

Everything below it is **vocabulary-agnostic**: the same loop serves LLM hidden streams, Qwen-Drive camera frames, and Flux latents.



The [UM] consumes host encodings through the encoder; ids are ingested into the bit plane, and ordered materialization restores ids to the host.

The channel model is $\text{conv}(n, \text{dim})$: n-grams are $\text{conv}(n, \text{dim}=1)$, grids $\text{conv}(n, \text{dim}=2)$, tensors $\text{conv}(n, \text{dim}=N)$ — shared 1×1 G1, $\text{seed}(n, \text{dim}) = (\text{dim}-1) \cdot 3 + (n-1)$.

ewm-state-machine — a Rust workspace in four bands

01 BIT PLANE

hllset-core — the HLLSet bit plane, the state representation · **hllset-lut** — the LUT lattice (token interpretation)
hllset-morphisms — ingest / materialize + default api (SHA1 + hllsetLUT) · **hllset-storage** — memory + sled content-addressed storage
hllset-contracts — soldered invariants (leaf) · **hllset-cid** — embedded SHA-1 CIDs

02 STATE STACK

ewm-git — the state stack: commit DAG, tip, recovery · **ewm-app** — the [UM] harness: stateless driver + StateCache
context-tree — Merkle tree over the working set (S(t) presentation) · **ewm-cache** — Arrow-backed extended cache

03 APPLICATIONS

ewm-scene — LLM ↔ HLLSet side-car helper (direct morphisms) · **ewm-flux-host** — Flux / MMDiT side-car host adapter
ewm-ops — the operational graph: values + programs + stack dispatcher · **ewm-sm-explore** — read-only explorer of the three-layer structure

04 TRAINER • PY

The trainer / controller reference implementation: **protocol** — probe config, trajectory, shapes · **predictors** — persistence / linear / DFT-periodic / ridge portfolio
selector — EWMA surprise + ϵ -greedy meta-controller · **adapters** — Synthetic / Llm · **loop** — open + closed (memory + curiosity)

Design notes live in docs/: [ARROW_CACHE](#) · [EXPLORER](#) · [SEPARATION](#) · [BOOLRING](#) · [BASIS_FRAMES](#) · [TRAINER](#) · [PODMAN](#) · [ASSIGNMENT_QWENDRIVE](#)

ewm-ops — an operational graph that boots like an OS

A **content-addressed operational graph** whose two sides are tied by directed edges:

h:<sha1> — the value lattice: HLLSets

p:<sha1> — the program lattice: expressions — any DSL expression is a [UM], persistence optional

v:<sha1> — the content-addressed vocabulary, compiled from the boot file

Traversal is a **stack-pop dispatcher**: fan-out by reference, deterministic fire sequence, feedback cycles under a fire budget.

The CLI boots it like an OS: compile the boot file, pick up the persisted state from the top of the stack, run the dispatcher, and commit the fire log into ewm-git.

```
$ cargo run -p ewm-ops -- \
  --store /tmp/ewm-ops-demo \
  --boot path/to/boot.ops \
  --fires 12 --repo .../repo

$ cargo run -p ewm-ops -- \
  --store /tmp/ewm-ops-demo
# picks up state on top of stack

$ cargo run -p ewm-ops log \
  --store /tmp/ewm-ops-demo
# append-only boot log

$ cargo run -p ewm-ops prev \
  --store /tmp/ewm-ops-demo
# roll latest back one boot
```

log / list / prev give the boot history: append-only log, distinct boots, and rollback of the latest boot.

Side-cars close the loop around real models

ewm-scene

LLM ↔ HLLSet side-car helper — direct morphisms

Makes the four ideas executable: `ingest` / `materialize` (morphism + fiber), `noether` / `subframes` (the D/R/N frame), `pyramid` (joined perceptrons), `sidecar` (the ring frame), `project` (any named frame).

ewm-flux-host

side-car host adapter for a rectified-flow MMDiT sampler — developed against a synthetic random-weight shim

One call closes the whole loop on a synthetic latent trajectory. Per denoising step the JSON report carries: `S(t)` popcount + content key, `D/R/N`, the Boolean-ring record (soft/hard key, residual, rotation), and the warning flags.

The two-score evaluation

Loop accuracy — ordered + set latent-token restoration, expected 1.0 · **Decode quality** — latent reconstruction cosine / MSE.

Because the ring basis changes as the run grows, the report also carries the lazy back-propagated view: `soft_final` / `step_final` re-project every step onto the final basis — each cached vector is stamped with the basis generation, and only the entries queried are recomputed.

The notebook is the application — the [UM] runs the cells

01-02

Foundations

Morphisms step by step — ingest (SHA1 + three pointers + hllsetLUT), materialize (ordered / no_order / beam), the [UM] loop and recovery; then the LLM ↔ HLLSet side-car rebuilt on ewm-scene.

03-05

Test benches

Qwen-Drive-1.0-4B perception tokens through the side-car loop; the perceptron pyramid (the union top perceptron u-HLLSet(t) and its three decompositions); the Flux Phase-1 bench on a synthetic MMDiT shim.

06-07

Graph & cooperation

The ewm-ops operational graph boots like an OS; then the ewm-jepa cooperation — V-JEPA encoder / predictor ↔ the Rust lattice (pipeline / IICA, three-LUT unification, [UM]-Net fan-out, holographic D/R/N).

08-12

Multi-LLM side-car loop

Three real small LLMs (DeepSeek-R1-Distill-Qwen-1.5B, Qwen2.5-1.5B-Instruct, gpt2) feed the union state $S(t) = \cup S_i(t)$; a DFT observer detects the query period, a controller feeds memory + curiosity back into the prompts, and a predictor portfolio with a meta-controller learns which predictor to trust.

Observed on the bench: a 16-step query cycle — $bss(t+16) = bss(t)$, rings plateau at dim 16; the EWMA selector still beats the learned selector on that loop (0.95 vs 1.11 meta/best).

Five products, one measurement lattice

All built on the same measurement lattice as the side-car — language models, world models, vision, document intelligence and vision-language-action autonomy.

NanoLM · Language Models

IMPLEMENTED

Megabyte-scale domain LLM built on De Bruijn Markov chains — deterministic, CPU-only, no GPU and no pretrained weights.

EWM Japa · World Models

PROTOTYPE

JEPA-style world-model memory service: frames encoded into a deterministic patch codebook, stored as content-addressed HLLSet keys, recalled by Bell State Similarity — no GPU, no torch, never a reconstruction of raw pixels.

Vision Studio · Vision

IN PROGRESS

Studio runtime for the vision side-car — 16×16 patch grids through conv(n,2) encoders, frame-by-frame positional restoration, with the lattice's DRN decomposition and temporal pyramid.

DeepSeek-OCR · Document Intelligence

IMPLEMENTED

Document extraction on DeepSeek-OCR with a CPU-only Tesseract fallback; drives the semantic compressor across a real 128K-vocabulary BPE boundary at 100% word retention and zero order loss.

EWM VLA · Autonomous Systems

PROTOTYPE

Vision-Language-Action side-car: visual embeddings, language tokens and action trajectories are mapped into HLLSet frames for content-addressable scene memory and lattice reasoning over perception-action chains — hash-level round-trip accuracy 1.0 on a real VLA host.

Partner with the research team: run the side-car loop against your own model as a design partner, pilot it on a real workload, or engage on a custom build.

BRING YOUR OWN MODEL — WE BRING THE MEMORY

The model stays. The memory is added.

A side-car, not a replacement — the host keeps its weights, tokenizer, prompts and APIs; nothing is retrained, fine-tuned or replaced.

A structure, not a summary — D / R / N per commit: what departed, what is retained, what is new, with a conservation balance.

Auditable by construction — content-addressed, immutable commits form an append-only record, on-premise inside your own boundary.

Portable across models — one measurement lattice spans hosts and modalities; the memory travels, the model stays where it is.

GET STARTED

```
$ git clone \
  https://github.com/alexmy21/ewm-state-machine

$ cargo run -p ewm-ops -- \
  --store /tmp/ewm-ops-demo \
  --boot path/to/boot.ops --fires 12

$ jupyter notebook \
  notebooks/01_ingest_materialize_um.ipynb
```

Design-partner pilots, evidence and docs:
codenticalai.com · support@codenticalai.com

$$H(t) = (S(t), H(t-1), D, R, N)$$

github.com/alexmy21/ewm-state-machine

